Limitless Deal Hub presentation tutorial

An English presentation and live demo in approximately eight minutes

Presenter: Durim Memishi

Target duration: 5 to 10 minutes

Recommended rehearsal target: 8 minutes including the live demo

Before the presentation

1. Open the slide deck and the live website in separate tabs. Use presenter notes if available.
2. Check https://comp-graph-project.vercel.app/api/health. It should report showroom-accounts and database ready.
3. Sign in with a demonstration account before presenting. Do not display passwords, connection strings, personal account records or provider dashboards.
4. Open Ferrari F8 Tributo at /cars/7#customize-car. Let its model finish loading. A controlled demo design can remain in this account.
5. Prepare /compare?cars=7,9,11 and the City Drive tab as fallbacks. For the designed vehicle, launch the drive through Test drive my design so it receives the chosen configuration.
6. Confirm the keyboard controls. Choose Lite graphics if the presentation computer struggles, and use a landscape viewport for a phone demonstration.
7. Rehearse once with a timer. Keep the browser and slides at a readable zoom. Close unrelated tabs and silence notifications.

If presenting locally

Open the project root in VS Code and run npm ci, then npm run dev. Keep the terminal running and use http://localhost:5173. Local accounts use a separate SQLite database, so a production account will not automatically exist locally. Prepare a local demonstration account in advance.

Running order

1. Title and project introduction: 30 seconds
2. Project aim and scope: 40 seconds
3. Website walkthrough: 45 seconds
4. Three.js implementation: 55 seconds
5. Customization and saving: 60 seconds
6. City driving: 60 seconds
7. Backend and database: 55 seconds
8. Live demonstration: 100 seconds
9. Evaluation and next steps: 35 seconds

Total: approximately 8 minutes. Pause for questions afterward rather than trying to fit a discussion into the timed demo.

Slides one to three

Slide one Limitless Deal Hub

Time: 0:00 to 0:30

Say: "My project is Limitless Deal Hub, an interactive car showroom built for Computer Graphics. A visitor can inspect a vehicle in 3D, customize its appearance and take that design into a controllable city scene. I will explain the graphics, the account system and the database, then show a short demonstration."

Show: The cover slide. Keep the introduction brief and move directly to the project's purpose.

Slide two Project aim and scope

Time: 0:30 to 1:10

Say: "The aim is to make vehicle exploration interactive. Instead of relying only on prepared photographs, users can rotate a model and observe changes to its materials and lighting. The current collection contains 12 vehicles from nine brands. The application is an educational prototype, so prices and checkout are demonstrations, and driving uses arcade physics."

Show: The scope slide. Explain what the application demonstrates without claiming measured commercial benefits or manufacturer-level vehicle accuracy.

Slide three Website walkthrough

Time: 1:10 to 1:55

Say: "The home page introduces the 3D model and the brand carousel. Users can filter the catalog by brand or powertrain, search for a car and change the sort order. A heart control saves a shortlist on this device. They can compare up to three cars before opening the customization page. Accounts add a profile, named saved designs and a persistent cart."

Show: The browsing steps on the slide. Mention that a device shortlist and a named account design are different types of saved information.

Transition

Say: "The main computer graphics work begins when a vehicle model enters the scene."

Slides four to seven

Slide four Three.js implementation

Time: 1:55 to 2:50

Say: "Three.js renders the scene. React Three Fiber connects it to React, and Drei supplies controls and scene utilities. Each catalog ID maps to a local GLB model. The application prepares its geometry, then applies lighting, camera controls and an assembly animation. Model materials are mapped to body, wheel and glass roles. The wheel rig creates four axle-centered pivots. Compressed models can contain integer attributes, so the preparation step converts positions and recomputed normals to floating-point data before transforming the geometry."

Emphasize: Meshes contain geometry and materials. Transformations place them in the scene. Lights, cameras and frame updates determine how the user sees and interacts with them.

Slide five Customization and saving

Time: 2:50 to 3:50

Say: "The design state keeps paint, finish, graphics, wheels, tint and studio lighting as separate fields. Changing wheels therefore preserves the body paint. The browser automatically saves a draft for each vehicle. A signed-in user can also name a design and save it through the API. That account record can later reopen the model or launch a drive. The cart stores a snapshot of the configuration, so two different designs of the same car remain separate items."

Emphasize: Browser drafts are convenient for local editing. Account designs and carts provide server persistence. The demonstration will show the same configuration entering the game.

Slide six City driving

Time: 3:50 to 4:50

Say: "The city uses keyboard or touch input to update a vehicle simulation. Small time steps make movement more consistent across frame rates and support collision checks. W and S control acceleration and reverse, A and D steer, and Space brakes. The wheel pivots rotate with speed and the front wheels steer. Traffic, pedestrians and signals animate the environment. Day and Night interpolate the sky and lighting while moving the sun and moon. Camera presets and a free view let the user inspect the car while driving."

Emphasize: Speed values are arcade settings. This implementation demonstrates graphics and interaction rather than a complete driving simulator.

Slide seven Backend and database

Time: 4:50 to 5:45

Say: "The frontend calls same-origin API routes for accounts, profiles, carts and saved designs. On my PC, a Node server creates a SQLite database automatically. In deployment, a Vercel function connects to Neon PostgreSQL. The main records are users, sessions, carts and designs. The backend derives password hashes with scrypt and identifies signed-in users through a session cookie. Queries restrict saved records to their owner. Local accounts and cloud accounts are separate, and signing out does not delete an account's saved cart or designs."

Show: The local and deployed setup on the slide. Keep credentials and database rows off the projector.

Slide eight live demonstration

Time: 5:45 to 7:25

Use the prepared demonstration account. Avoid creating a new account or downloading a receipt during the timed section.

Step one browse and compare

About 15 seconds. Open the home collection and point out the new brands. Open the prepared comparison link for Ferrari, Bugatti and Pagani. Say: "The collection supports filtering and comparison before choosing a model."

Step two customize the Ferrari

About 25 seconds. Open /cars/7#customize-car. Choose a clear body color, then select Gold in Wheel finish. Point out that the body color remains selected. Change the lighting preset once. Say: "These material roles update independently, and lighting changes how the same surface appears."

Step three save the design

About 15 seconds. Use a short name such as Presentation Ferrari and select Save to my profile. Wait for its success message. Add the design to the cart only if time permits. Say: "This named configuration belongs to my account. The browser draft is a separate local copy."

Step four drive the same configuration

About 35 seconds. Choose Test drive my design. Wait for Start / resume drive to become enabled, then start. Hold W briefly, steer with A or D, release, and brake with Space. Switch to Left or Right view while safe, then choose Night to show the transition. Use Pause before returning to the slides.

Say: "The game receives the selected design. Wheel movement, camera changes and the day and night transition update in real time."

Step five return to the profile

About 10 seconds. If the scene loaded quickly, exit the drive, open Profile and show the saved entry. If time is short, point to the already visible save confirmation and return to slide nine. Do not rush to perform extra steps.

Slide nine evaluation and next steps

Time: 7:25 to 8:00

Say: "The baseline passed 13 automated regression checks covering the cart, catalog, material state, driving updates and wheel geometry. All six new models also passed integration checks. The next priorities are broader device performance measurements, API integration tests and account recovery. Third-party models retain their attribution, and the prototype explains its demo limits. The main result is a working connection between web interaction, 3D graphics and persistent user designs."

End: Leave the project link visible and invite questions.

Fallbacks and rehearsal

Five minute version

Use the same nine slides. Keep the opening to 15 seconds, spend about 25 seconds each on slides two and three, 35 seconds on slide four, 30 seconds each on slides five to seven, 80 seconds on the demo and 15 seconds on the closing. The resulting talk is approximately 4 minutes 45 seconds, leaving a small buffer for switching tabs and loading.

If the network fails

Use the prepared local project and local demonstration account if available. Otherwise continue with the deck's actual catalog images and explain the workflow. Do not claim a live action succeeded if the request failed. Describe which features require the backend and which 3D features run in the browser.

If a model loads slowly

Keep the loading tab open rather than repeatedly refreshing. Use the preloaded model or Lite graphics. A simplified demonstration of independent paint and wheel finish is more useful than waiting for several different cars to load.

If the account server fails

Check /api/health before the talk. During the talk, demonstrate browser customization and explain that named account saving requires the server. Avoid showing provider credentials or attempting database repairs on the projector.

Likely questions and concise answers

Why Three.js and React Three Fiber

Three.js provides the graphics objects and renderer. React Three Fiber integrates those scenes into the same component and state model as the website. Drei supplies common utilities, reducing repeated setup code.

Did you create all the vehicle models

The vehicle assets come from credited third-party artists. The project adapts them for loading, material editing, assembly animation and wheel movement. Each source and license appears in the model credits.

What survives logout

Account designs and carts stay in the server database and return after signing into the same account. The interface switches to the separate guest cart after logout. Browser drafts and shortlist selections remain on that device unless its storage is cleared.

Is the game physically realistic

It has time-stepped movement, steering, braking and collision checks, but it uses arcade limits and simplified behavior. It does not reproduce manufacturer testing or a full vehicle dynamics model.

What would you improve next

Add account recovery and email verification, broaden API integration and accessibility checks, measure performance on physical devices and introduce reviewed database migrations. Preserve source model licenses before considering commercial use.

Presenter reference

Live site: https://comp-graph-project.vercel.app/ . The prepared comparison uses /compare?cars=7,9,11 and the Ferrari studio uses /cars/7#customize-car.

Source: https://github.com/Dm31650/comp-graph-project